



Software Testing and Quality Assurance (05302438)

**Yadagiri Rama Deepak,
Assistant Professor
Computer Science & Applications**



Module-5

Software Quality and Reliability

- Software quality Introduction,
- Definition,
- Software quality- quality control,
- Quality assurance,
- Software reliability introduction,
- Measures,
- Models,
- Software Safety.

Software quality Introduction

What is Software Quality?

Software Quality shows how good and reliable a product is. To convey an associate degree example, think about functionally correct software. It performs all functions as laid out in the SRS document. But, it has an associate degree virtually unusable program. even though it should be functionally correct, we tend not to think about it to be a high-quality product.

Another example is also that of a product that will have everything that the users need but has an associate degree virtually incomprehensible and not maintainable code. Therefore, the normal construct of quality as “fitness of purpose” for code products isn't satisfactory.

Software quality Introduction

Definition:

◆ Software Quality

The degree to which software meets specified **functional and non-functional requirements**, is **free of defects**, and satisfies **user expectations**.

◆ Software Reliability

The **probability** that software will **function correctly** without failure for a specified period of time under specified conditions.



Quality Assurance (QA)

Quality Assurance (QA)

Quality Assurance (QA) and Quality control (QC) are both important methods in software engineering to get high-quality software. Quality Assurance (QA) prevents software defects or minimizes the number of defects in software before delivery by making sure that proper methods and processes are followed during the software development process. Whereas Quality Control (QC) identifies and fixes the defects or errors that exist after development.



Quality Assurance (QA)

What is Quality Assurance (QA)?

Quality assurance is a method of making the software application with fewer defects and mistakes when it is finally released to the end users.

Quality Assurance is defined as an activity that ensures the approaches, techniques, methods, and processes designed for the projects are implemented correctly. It recognizes defects in the process. Quality Assurance is completed before Quality Control. It focuses on preventing defects.

It is a proactive process and is preventive in nature.

It helps to recognize flaws in the process.

These activities monitor and verify that the processes used to manage and create deliverables have been followed.



1. Software Quality: Key Aspects

Aspect

Functionality

Usability

Efficiency

Maintainability

Portability

Security

Description

Correctness, suitability, and accuracy of the software's features.

Ease of use, user interface quality, and user experience.

Use of system resources like CPU and memory.

Ease of modifying, fixing bugs, or upgrading the software.

Ability to run across different environments or platforms.

Protection against unauthorized access or data breaches.

Quality Standards:

- **ISO 25010:** Standard model for software product quality.
- **CMMI** (Capability Maturity Model Integration)
- **IEEE 730:** Software Quality Assurance Plan

2. Software Reliability: Key Concepts

Concept

Mean Time Between Failures (MTBF)

Mean Time To Repair (MTTR)

Failure Rate

Availability

Description

Average time between system failures.

Time taken to fix a failure and restore service.

Number of failures per time unit or usage unit.

Percentage of time the system is operational.



Relationship Between Quality and Reliability:

Factor	Software Quality	Software Reliability
Focus	Meeting requirements & user satisfaction	Consistent, error-free performance over time
Measured By	Functionality, usability, maintainability	MTBF, availability, failure rate
Impacts	Development process, user acceptance	Long-term use, trust, downtime risk



Improving Software Quality & Reliability:

Technique	Description
Code Reviews	Peer review of code to detect early defects.
Automated Testing	Unit, integration, and regression tests run regularly.
Continuous Integration/Deployment (CI/CD)	Ensures smooth, repeatable, and reliable updates.
Static Code Analysis	Detect coding errors before runtime.
Defect Tracking	Systematic recording and analysis of bugs.
Reliability Testing	Simulate stress, load, and error conditions.

Measures

Definition

Measures are quantitative values collected from the software development and testing process.

They represent raw data about project, process, or product attributes.

Types of Measures

1. Process Measures

Related to how testing is performed.

Examples:

Number of test cases executed

Test coverage (requirements, functions, code)

Defect detection efficiency

2. Product Measures

Related to the quality of the final software product.

Examples:

Defect density (defects per KLOC or per function point)

Mean Time to Failure (MTTF)

Reliability & availability

Types of Measures

3. Project Measures

Related to project management and resource tracking.

Examples:

Effort variance (planned vs. actual)

Schedule variance

Cost of quality

Types of Measures

Measures vs. Metrics

Measure = raw number (e.g., *20 defects*).

Metric = analyzed value derived from measures (e.g., *Defect Density = 20 defects / 1 KLOC*).

Measures

Measures vs. Metrics

Measure = raw number (e.g., *20 defects*).

Metric = analyzed value derived from measures (e.g., *Defect Density = 20 defects / 1 KLOC*).

Measures

Importance of Measures in STQA

Track **progress** of testing.

Identify **defects early** and improve detection.

Provide **evidence of quality** to stakeholders.

Support **decision-making** for process improvements.

Monitor **cost, schedule, and effort efficiency**.

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in

